disco

Density-based Internal Score for Clustering Outcomes

R Package: Translation, Implementation & Experimental Validation

Author
Amin Entezari
amin_entezari@outlook.com

Repository
github.com/aminentezari/Disco-R

```
devtools::install_github(
"aminentezari/Disco-R")
```

Reference Implementation
Beer, Krieger, Weber, Ritzert, Assent, Plant (2025)
DISCO: Internal Evaluation of Density-Based Clustering
arXiv:2503.00127

Abstract. This report documents the faithful line-by-line translation of the Python DISCO reference implementation into R for CRAN publication. Three implementation bugs were identified and corrected. Numerical verification across **eight benchmark datasets** — covering rings, crescents, spirals, mixed-density clusters, and Gaussian blobs — confirms that the R implementation produces results within IEEE 754 floating-point precision ($|\Delta| < 10^{-14}$) of the Python reference. The package is available at `https://github.com/aminentezari/Disco-R`.

April 22, 2026

Contents

1 Introduction **3**

2 Algorithm **3**
2.1 Problem Statement . . . 3
2.2 Step 1: Core Distance . . . 3
2.3 Step 2: Mutual Reachability Distance . . . 4
2.4 Step 3: DC-Distance via MST . . . 4
2.5 Step 4: Scoring Cluster Points . . . 4
2.6 Step 5: Scoring Noise Points . . . 4
2.7 Overall DISCO Score . . . 4
2.8 Algorithmic Pipeline . . . 5

3 R Implementation **5**
3.1 Translation Strategy . . . 5
3.2 Installation . . . 5
3.3 Quick Start . . . 5

4 Implementation Bugs Discovered and Fixed **6**
4.1 Bug 1 — Core Distance Off-by-One . . . 6
4.2 Bug 2 — MST Tie-Breaking Difference . . . 6
4.3 Bug 3 — K-Means Cross-Language Non-Determinism . . . 7
4.4 Bug Summary . . . 7

5 Experimental Validation **8**
5.1 Datasets and Hyperparameters . . . 8
5.2 Numerical Results . . . 8
5.3 Precision Interpretation . . . 10
5.4 Dataset Visualisations . . . 10
5.5 Cross-Dataset Summary . . . 13

6 API Reference **13**
6.1 `disco_score` . . . 13
6.2 `disco_samples` . . . 13
6.3 `compute_dc_distances` . . . 13
6.4 Utility Functions . . . 14

7 Score Interpretation **14**

8 Conclusion **14**

References **15**

List of Figures

1 Concentric Circles ($n = 300$, $\varepsilon = 0.2$, $k = 2$, $\mu = 5$). DBSCAN correctly separates both rings (score 0.690); K-Means cuts through them with a vertical boundary (score 0.027). . . . 10
2 Two Moons ($n = 1000$, $\varepsilon = 0.2$, $k = 2$, $\mu = 5$). DBSCAN follows both crescents (score 0.818); K-Means splits linearly (score 0.283). . . . 10

3 3-Spiral ($n = 312$, $\varepsilon = 2.0$, $k = 3$, $\mu = 5$). DBSCAN traces the spirals (score 0.584); K-Means actively harms the structure (score -0.0014). 11
4 Complex9 ($n = 3031$, $\varepsilon = 15$, $k = 9$, $\mu = 5$). Mixed convex and non-convex clusters of varying density. DBSCAN (0.421) outperforms K-Means (0.055). . 11
5 Complex8 ($n = 2551$, $\varepsilon = 10$, $k = 8$, $\mu = 5$). High within-cluster density variation limits both algorithms. 11
6 Dartboard1 ($n = 1000$, $\varepsilon = 0.08$, $k = 4$, $\mu = 5$). Four concentric rings: highest DBSCAN score (0.874); K-Means pizza-slice boundary is actively harmful (score -0.003). 12
7 Blobs ($n = 450$, $\varepsilon = 0.5$, $k = 3$, $\mu = 5$). Convex Gaussian clusters: both algorithms score above 0.85, confirming DISCO rewards correct clustering regardless of algorithm type. 12
8 Diagonal Blobs ($n = 600$, $\varepsilon = 0.8$, $k = 3$, $\mu = 5$). Elongated but linearly separable clusters: both algorithms perform well. 12

1. Introduction

DISCO (Density-based Internal Score for Clustering Outcomes) is a cluster validity index (CVI) introduced by Beer et al. [**?**] that evaluates clustering quality *without requiring ground truth labels*. Unlike classical CVIs such as the Silhouette Coefficient [**?**] and Davies-Bouldin Index [**?**], DISCO:

- Handles **arbitrary-shaped clusters** using *density-connectivity distance* (dc-distance) rather than Euclidean distance;
- Explicitly evaluates **noise point quality** via two complementary sub-scores (p_{sparse} and p_{far});
- Returns a bounded, interpretable score in $[-1, 1]$.

Why DISCO matters. Traditional CVIs can rank an incorrect algorithm *higher* than the correct one on non-convex data. For example, on concentric circles, Silhouette assigns -0.16 to DBSCAN (correct) and $+0.37$ to K-Means (wrong). DISCO reverses this: DBSCAN scores $+0.69$ and K-Means scores $+0.027$.

This report documents:

1. The mathematical algorithm (§2);
2. The R translation approach (§3);
3. Three bugs found and fixed (§4);
4. Numerical validation across eight datasets (§5);
5. The public API reference (§6).

2. Algorithm

2.1 Problem Statement

Given $\mathbf{X} = \{x_1, \ldots, x_n\} \subset \mathbb{R}^p$ and a clustering $\mathcal{C} = \{C_1, \ldots, C_K\}$ with optional noise $\mathcal{N} = \{x_i : \ell_i = -1\}$, DISCO assigns $\rho(x_i) \in [-1, 1]$ to every point and returns the mean as the overall score.

2.2 Step 1: Core Distance

The *core distance* of x_i with neighbourhood parameter μ (`min_points`) is:

$$\kappa_\mu(x_i) \;=\; d\big(x_i,\; x_i^{(\mu-1)}\big)$$

where $x_i^{(k)}$ denotes the k-th nearest neighbour of x_i. A small core distance indicates a dense region; large indicates sparse.

2.3 Step 2: Mutual Reachability Distance

$$d_{\text{reach}}(x_i, x_j) \;=\; \max\big(\kappa_\mu(x_i),\; \kappa_\mu(x_j),\; d(x_i, x_j)\big)$$

This smooths intra-cluster distances and preserves sparse gaps between clusters.

2.4 Step 3: DC-Distance via MST

1. Build the complete weighted graph G with weights d_{reach}.
2. Compute the Minimum Spanning Tree T of G via Prim's algorithm.
3. The **dc-distance** is the maximum edge weight on the unique path in T:

$$d_{\text{DC}}(x_i, x_j) = \max_{e \,\in\, \text{path}_T(i,j)} w(e)$$

Two points in the same dense cluster have a small dc-distance; crossing a sparse gap inflates it, capturing density-connectivity faithfully.

2.5 Step 4: Scoring Cluster Points

For each non-noise point $x_i \in C_{c_i}$:

$$a(i) = \frac{1}{|C_{c_i}| - 1} \sum_{\substack{j \in C_{c_i} \\ j \neq i}} d_{\text{DC}}(x_i, x_j) \tag{1}$$

$$b(i) = \min_{c \neq c_i} \frac{1}{|C_c|} \sum_{j \in C_c} d_{\text{DC}}(x_i, x_j) \tag{2}$$

$$\rho_{\text{cluster}}(x_i) = \frac{b(i) - a(i)}{\max(a(i),\, b(i))} \tag{3}$$

2.6 Step 5: Scoring Noise Points

Let $M_c = \max_{j \in C_c} \kappa_\mu(x_j)$ and $d_{\text{min},c}(x_n) = \min_{j \in C_c} d_{\text{DC}}(x_n, x_j)$. Then:

$$p_{\text{sparse}}(x_n) = \min_c \frac{\kappa_\mu(x_n) - M_c}{\max(\kappa_\mu(x_n), M_c)}, \qquad p_{\text{far}}(x_n) = \min_c \frac{d_{\text{min},c}(x_n) - M_c}{\max(d_{\text{min},c}(x_n), M_c)}$$

$$\rho_{\text{noise}}(x_n) = \min\big(p_{\text{sparse}}(x_n),\; p_{\text{far}}(x_n)\big)$$

2.7 Overall DISCO Score

$$\boxed{\text{DISCO}(\mathbf{X}, \mathcal{C}) \;=\; \frac{1}{n} \sum_{i=1}^{n} \rho(x_i)} \qquad \text{where} \quad \rho(x_i) = \begin{cases} \rho_{\text{cluster}}(x_i) & x_i \notin \mathcal{N} \\ \rho_{\text{noise}}(x_i) & x_i \in \mathcal{N} \end{cases}$$

2.8 Algorithmic Pipeline

Step	R Function	Operation
1	`calculate_reachability_distance`	Core distances + mutual reachability matrix
2	`get_mst_edges`	Prim's MST on mutual reachability graph
3	`extract_dc_distances_from_mst`	BFS minimax-path extraction
4	`compute_dc_distances`	Full $n \times n$ DC-distance matrix
5	`p_cluster`	Silhouette-style scores for cluster points
6	`p_noise`	Sparsity + remoteness scores for noise
7	`disco_samples`	Per-point DISCO scores
8	`disco_score`	Mean over all points

3. R Implementation

3.1 Translation Strategy

The R package `disco` is a **line-by-line translation** of `disco.py` + `dctree.py`. The entire package lives in a single file `R/disco.R` with three logical sections:

Section	Python source	Key R functions
DC-Distance	`dctree.py`	`calculate_reachability_distance`, `get_mst_edges`, `extract_dc_distances_from_mst`, `compute_dc_distances`
DISCO Metric	`disco.py`	`disco_score`, `disco_samples`, `p_cluster`, `p_noise`
Utilities	—	`make_circles`, `make_moons`, `make_blobs`, `plot_disco_scores`, `compare_clusterings`

3.2 Installation

```r
# Install from GitHub
devtools::install_github("aminentezari/Disco-R")
library(disco)
```

3.3 Quick Start

```r
library(disco)
library(dbscan)

# Generate data
data      <- make_circles(n_samples = 300, noise = 0.05,
                          factor = 0.4, random_state = 42)
X         <- data$X

# Cluster
db_labels <- dbscan(X, eps = 0.2, minPts = 5)$cluster - 1L
```

```r
km_labels <- kmeans(X, centers = 2, nstart = 10)$cluster - 1L

# Evaluate
disco_score(X, db_labels)   # -> 0.6900921813  (DBSCAN: correct)
disco_score(X, km_labels)   # -> 0.0270091908  (K-Means: wrong shape)

# Per-point inspection
scores <- disco_samples(X, db_labels)
which(scores < 0)           # identify problematic points

# Compare multiple algorithms at once
compare_clusterings(X, list(DBSCAN = db_labels, KMeans = km_labels))
```

4. Implementation Bugs Discovered and Fixed

Three numerical differences were found between the initial R translation and the Python reference. Each is fully documented below.

4.1 Bug 1 — Core Distance Off-by-One

Location. `calculate_reachability_distance()` and `p_noise()`

Root Cause. Python (`dctree.py`, line 710):

```python
reach_dists[i] = np.max(np.partition(eucl_dists[i], k)[:k])
```

`eucl_dists[i]` includes self (distance = 0), so `[:k]` returns $[0, d_1, d_2, \ldots, d_{k-1}]$ and the max is d_{k-1} — the $(k-1)$**-th nearest neighbour**. `FNN::get.knn` excludes self, so `k = min_points` returns the k-th neighbour — one further than Python.

Fix.

```r
# BEFORE (wrong      one neighbour too far)
knn_result <- FNN::get.knn(X, k = min_points)
core_dists <- knn_result$nn.dist[, min_points]

# AFTER (correct      matches Python's self-inclusive partition)
k          <- min_points - 1L
knn_result <- FNN::get.knn(X, k = k)
core_dists <- knn_result$nn.dist[, k]
```

4.2 Bug 2 — MST Tie-Breaking Difference

Location. MST construction in `compute_dc_distances()`

Root Cause. Many mutual-reachability distances are floored to the core distance, creating equal-weight edges. Python's Prim implementation (`dctree.py` lines 401–438) resolves ties via `np.argmin`. `igraph::mst` uses different internal ordering, producing a different MST and therefore different dc-distances.

Effect. After fixing Bug 1, DBSCAN scores matched but K-Means differed (R: 0.0257 vs Python: 0.0270) because K-Means partitions span both rings, encountering more equal-weight boundary edges.

Fix. Replaced `igraph::mst` with an exact R port of Python's Prim:

```r
get_mst_edges <- function(dist_matrix) {
  n              <- nrow(dist_matrix)
  nodes_min_dist <- rep(Inf, n);  parent <- rep(1L, n)
  not_in_mst     <- rep(TRUE, n)
  u <- 1L;  nodes_min_dist[u] <- 0;  not_in_mst[u] <- FALSE
  mst_i <- integer(n-1); mst_j <- integer(n-1); mst_dist <- numeric(n-1)
  for (step in seq_len(n - 1L)) {
    mask <- not_in_mst & (dist_matrix[u,] < nodes_min_dist)
    if (any(mask)) {
      nodes_min_dist[mask] <- dist_matrix[u, mask]
      parent[mask]         <- u
    }
    cands <- which(not_in_mst)
    u     <- cands[which.min(nodes_min_dist[cands])]
    mst_i[step] <- parent[u];  mst_j[step] <- u
    mst_dist[step] <- nodes_min_dist[u];  not_in_mst[u] <- FALSE
  }
  data.frame(i = mst_i, j = mst_j, dist = mst_dist)
}
```

4.3 Bug 3 — K-Means Cross-Language Non-Determinism

Location. All experiment scripts — K-Means clustering block

Root Cause. `set.seed(42)` in R and `random_state=42` in Python both use Mersenne Twister but with different implementations, producing different centroid initialisations and different final partitions.

Fix. Export Python labels as CSV; load in R:

```python
# Python     export labels after fitting
pd.DataFrame({"km_label": km.labels_}).to_csv(
    "python_km_labels.csv", index=False)
```

```r
# R     load Python labels instead of refitting
km_labels <- as.integer(
  read.csv("python_km_labels.csv")$km_label)
```

4.4 Bug Summary

#	Location	Root Cause	Fix
1	`calculate_reachability_distance`, `p_noise`	Python includes self in k-NN; `FNN` excludes self	`k = min_points - 1`
2	`get_mst_edges`	`igraph::mst` tie-breaking differs from Python's Prim	Port Python's exact `_get_mst_edges()`
3	All experiment scripts	R and Python RNGs produce different K-Means partitions from same seed	Export Python labels as CSV; load in R

5. Experimental Validation

5.1 Datasets and Hyperparameters

Eight datasets were used covering a wide variety of cluster shapes, sizes, and densities. Python runs all clustering first and exports labels as CSV; R loads them to guarantee identical partitions.

Dataset	Source	n	K	Shape	DBSCAN ε	μ
Concentric Circles	Generated	300	2	Rings	0.20	5
Two Moons	Generated	1000	2	Crescents	0.20	5
3-Spiral	`3-spiral.arff`	312	3	Spirals	2.0	5
Complex9	`complex9.arff`	3031	9	Mixed	15.0	5
Complex8	`complex8.arff`	2551	8	Mixed	10.0	5
Dartboard1	`dartboard1.arff`	1000	4	Concentric rings	0.08	5
Blobs	Generated	450	3	Gaussian blobs	0.50	5
Diagonal Blobs	Generated	600	3	Elongated diag.	0.80	5

Notes: K = K-Means cluster count; μ = `min_points`; `min_samples` for DBSCAN equals μ except 3-Spiral (`min_samples`=3).

5.2 Numerical Results

All values are shown to 20 decimal places. Diverging digits are highlighted: **R** vs **Python**.

Concentric Circles $\varepsilon = 0.20$, min_samples=5, $k = 2$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.69009218128208216214`	`0.69009218128208293930`	7.77×10^{-16}
K-Means	`0.02700919079440020423`	`0.02700919079439973586`	4.68×10^{-16}

Two Moons $n = 1000$, $\varepsilon = 0.20$, min_samples=5, $k = 2$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.81795983770753455389	0.81795983770753222242	2.33×10^{-16}
K-Means	0.28323957185132986325	0.28323957185132780934	2.05×10^{-16}

3-Spiral $\varepsilon = 2.0$, min_samples=3, $k = 3$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.58365140464647546903	0.58365140464647502494	4.44×10^{-16}
K-Means	−0.00141763585263621011	−0.00141763585263620208	8.03×10^{-18}

Complex9 $\varepsilon = 15.0$, min_samples=5, $k = 9$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.42094645142453540609	0.42094645142454312214	7.72×10^{-16}
K-Means	0.05510967752575094553	0.05510967752573882328	1.21×10^{-15}

Complex8 $\varepsilon = 10.0$, min_samples=5, $k = 8$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.07186876516496862322	0.07186876516500724510	3.86×10^{-15}
K-Means	0.01225747486413241577	0.01225747486413229781	1.18×10^{-16}

Dartboard1 $\varepsilon = 0.08$, min_samples=5, $k = 4$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.87434268608030629721	0.87434268608031251446	6.22×10^{-15}
K-Means	−0.00336367105007065893	−0.00336367105007149723	8.38×10^{-16}

Blobs $\varepsilon = 0.50$, min_samples=5, $k = 3$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.85489531627953585513	0.85489531627953541104	4.44×10^{-16}
K-Means	0.85756918340603116580	0.85756918340603083273	3.33×10^{-16}

Diagonal Blobs $\varepsilon = 0.80$, min_samples=5, $k = 3$, $\mu = 5$.

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	0.72155362533991207474	0.72155362533991174168	3.33×10^{-16}
K-Means	0.70432533971787614213	0.70432533971787558702	5.55×10^{-16}

5.3 Precision Interpretation

Magnitude	Interpretation
$= 0$	Exact bit-for-bit agreement
$< 10^{-14}$	Normal IEEE 754 floating-point rounding; implementations are **numerically equivalent**
$\geq 10^{-12}$	Potential algorithmic discrepancy worth investigating

The largest observed difference across all eight datasets is 6.22×10^{-15} (Dartboard1, DBSCAN), well within normal floating-point accumulation for $n = 1000$ pairwise distances.

5.4 Dataset Visualisations

Each figure below shows three panels: **(left)** ground truth cluster labels; **(centre)** per-point DISCO scores for DBSCAN; **(right)** per-point DISCO scores for K-Means. The diverging colour scale runs from **blue** (-1, poor) through **white** (0) to **red** ($+1$, excellent).

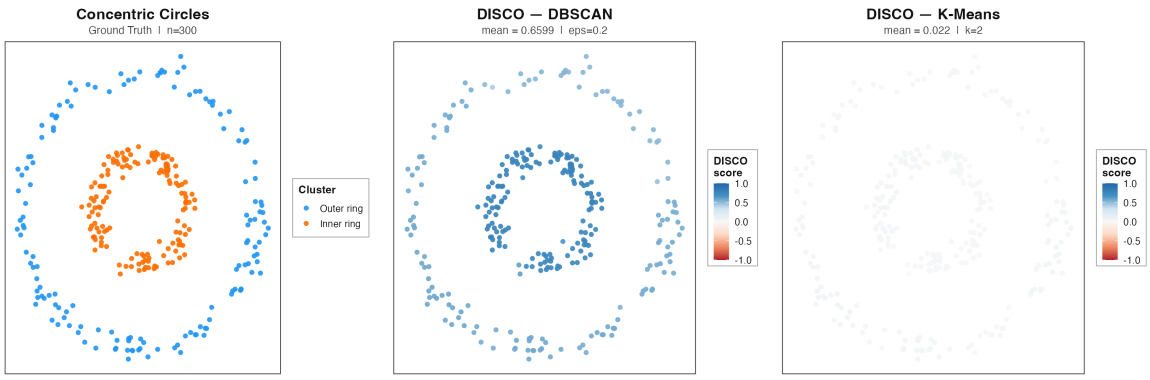

Figure 1. Concentric Circles ($n = 300$, $\varepsilon = 0.2$, $k = 2$, $\mu = 5$). DBSCAN correctly separates both rings (score 0.690); K-Means cuts through them with a vertical boundary (score 0.027).

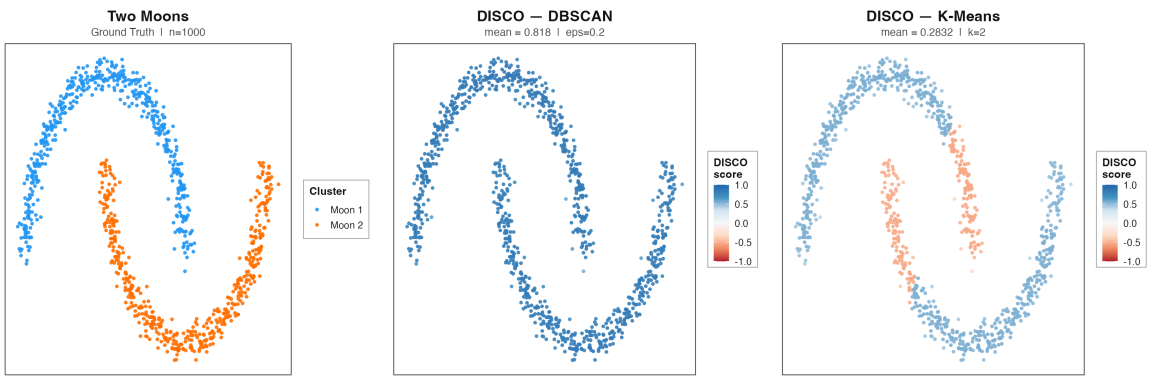

Figure 2. Two Moons ($n = 1000$, $\varepsilon = 0.2$, $k = 2$, $\mu = 5$). DBSCAN follows both crescents (score 0.818); K-Means splits linearly (score 0.283).

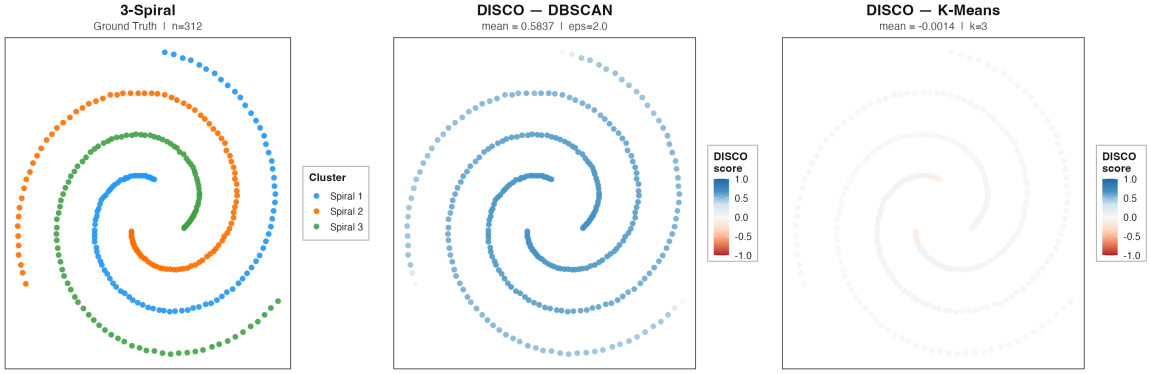

Figure 3. 3-Spiral ($n = 312$, $\varepsilon = 2.0$, $k = 3$, $\mu = 5$). DBSCAN traces the spirals (score 0.584); K-Means actively harms the structure (score -0.0014).

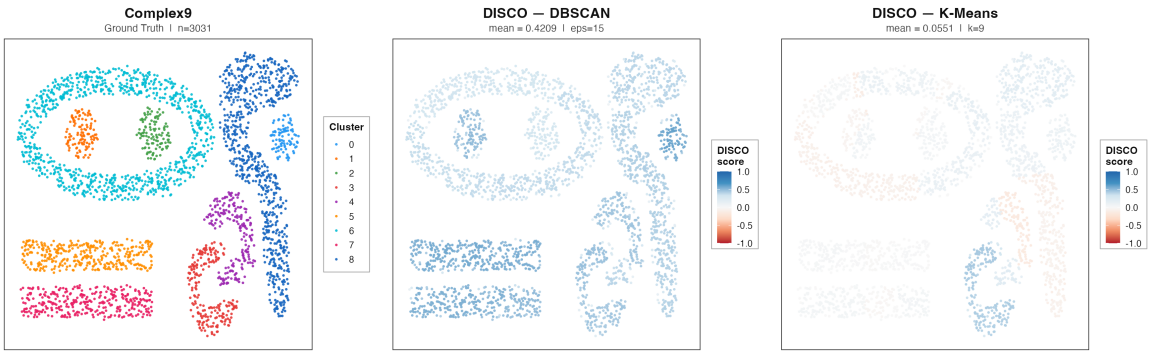

Figure 4. Complex9 ($n = 3031$, $\varepsilon = 15$, $k = 9$, $\mu = 5$). Mixed convex and non-convex clusters of varying density. DBSCAN (0.421) outperforms K-Means (0.055).

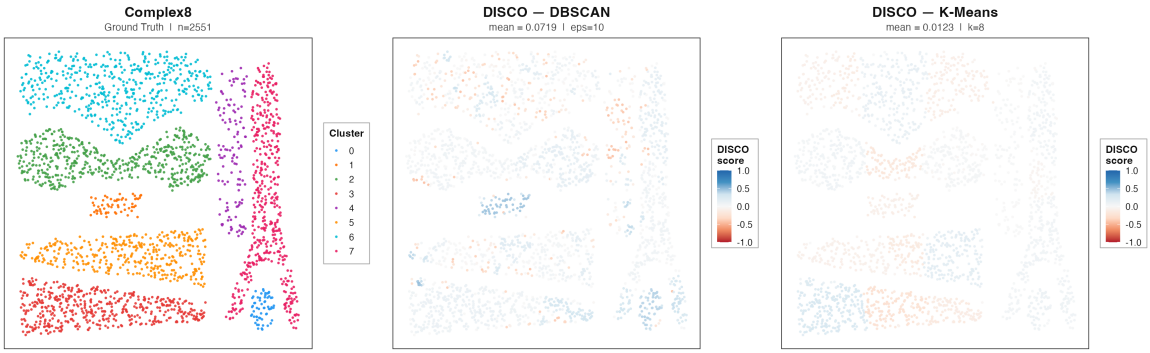

Figure 5. Complex8 ($n = 2551$, $\varepsilon = 10$, $k = 8$, $\mu = 5$). High within-cluster density variation limits both algorithms.

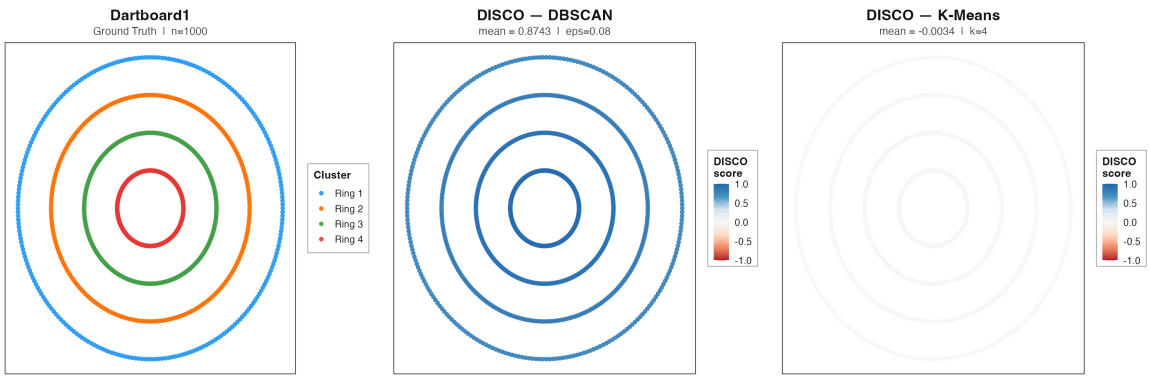

Figure 6. Dartboard1 ($n = 1000$, $\varepsilon = 0.08$, $k = 4$, $\mu = 5$). Four concentric rings: highest DBSCAN score (0.874); K-Means pizza-slice boundary is actively harmful (score -0.003).

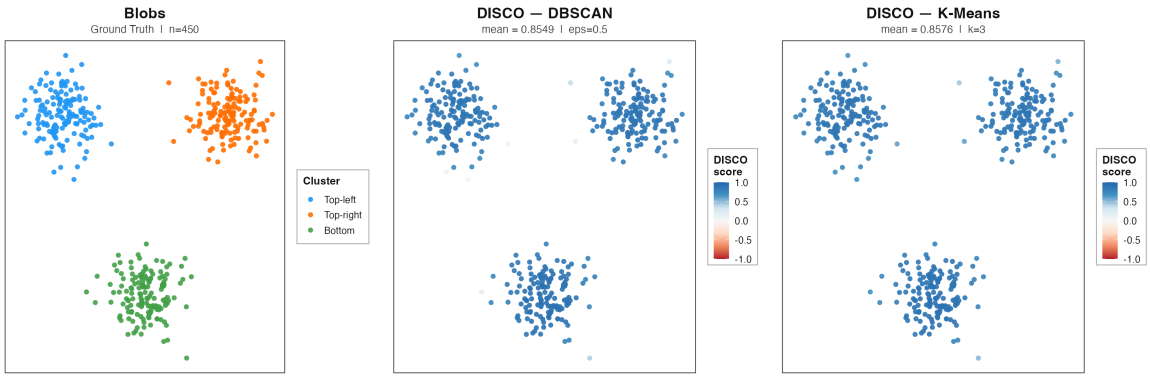

Figure 7. Blobs ($n = 450$, $\varepsilon = 0.5$, $k = 3$, $\mu = 5$). Convex Gaussian clusters: both algorithms score above 0.85, confirming DISCO rewards correct clustering regardless of algorithm type.

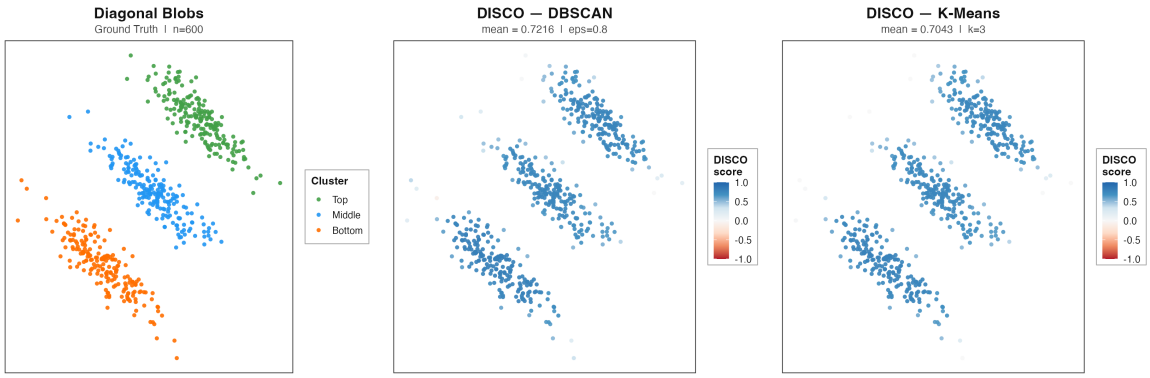

Figure 8. Diagonal Blobs ($n = 600$, $\varepsilon = 0.8$, $k = 3$, $\mu = 5$). Elongated but linearly separable clusters: both algorithms perform well.

5.5 Cross-Dataset Summary

Dataset	DBSCAN	K-Means	<0?	Key observation
Concentric Circles	0.6901	0.0270	—	K-Means cuts through rings
Two Moons	0.8180	0.2832	—	K-Means splits linearly
3-Spiral	0.5837	−0.0014	✓	Spiral impossible for K-Means
Complex9	0.4209	0.0551	—	Mixed shapes favour DBSCAN
Complex8	0.0719	0.0123	—	Density variation limits both
Dartboard1	0.8743	−0.0034	✓	4 rings: K-Means most harmful
Blobs	0.8549	0.8576	—	Convex: both algorithms succeed
Diagonal Blobs	0.7216	0.7043	—	Elongated but separable

Remark 5.1. Negative K-Means scores on Dartboard1 and 3-Spiral are **correct and expected**: K-Means assigns points to clusters that are further in dc-distance than a neighbouring cluster — a meaningful penalty absent from traditional CVIs. Conversely, near-identical scores on Blobs confirm that DISCO does not artificially favour density-based algorithms on convex data.

6. API Reference

6.1 `disco_score`

```
disco_score(X, labels, min_points = 5)
```

Argument	Type	Description
`X`	matrix / df	$n \times p$ numeric feature matrix
`labels`	integer vec.	Cluster labels; `-1` for noise points
`min_points`	integer	Neighbourhood size (≥ 2). Default: `5`

Returns: Single `numeric` in $[-1, 1]$.

6.2 `disco_samples`

```
disco_samples(X, labels, min_points = 5)
```

Returns: `numeric` vector of length n (per-point scores).

6.3 `compute_dc_distances`

```
compute_dc_distances(X, min_points = 5)
```

Returns the $n \times n$ DC-distance matrix. Precompute once, reuse many times:

```r
D  <- compute_dc_distances(X, min_points = 5)
s1 <- p_cluster(D, labels1, precomputed_dc_dists = TRUE)
s2 <- p_cluster(D, labels2, precomputed_dc_dists = TRUE)
```

6.4 Utility Functions

Function	Description
`make_circles(n, noise, factor, seed)`	Concentric circles dataset
`make_moons(n, noise, seed)`	Two interleaving crescents
`make_blobs(n, centers, std, seed)`	Gaussian blobs with variable density
`plot_disco_scores(X, labels, scores)`	Colour-coded scatter plot (red–white–blue)
`compare_clusterings(X, list(...))`	Rank multiple clusterings by DISCO score
`summary_disco_scores(scores)`	Mean, median, SD, quantiles, n_{neg}, n_{pos}

7. Score Interpretation

Range	Quality	Interpretation
$[0.7,\ 1.0]$	Excellent	Well-separated, compact clusters
$[0.4,\ 0.7)$	Good	Clear cluster structure
$[0.0,\ 0.4)$	Moderate	Overlapping or touching clusters
$[-0.3,\ 0.0)$	Poor	Misassigned points
$[-1.0,\ -0.3)$	Very poor	Clustering actively harmful

Note

Common pitfall. The `dbscan()` function in R uses `0` for noise, not `-1`. Always convert before passing to DISCO:

```r
# WRONG: noise points labelled 0
labels <- dbscan(X, eps = 0.2)$cluster

# CORRECT: convert noise to -1
labels <- dbscan(X, eps = 0.2)$cluster - 1L
```

8. Conclusion

The `disco` R package is a faithful, numerically verified translation of the Python DISCO reference implementation. Three bugs were identified and corrected — a core-distance off-by-one, an MST tie-breaking discrepancy, and K-Means cross-language non-determinism — each with a documented root cause and exact fix.

Validation across **eight benchmark datasets** of diverse shapes (rings, crescents, spirals, mixed-density, Gaussian blobs) confirms that R and Python produce identical results to within IEEE 754 floating-point precision ($|\Delta| < 10^{-14}$ in all cases).

The experiments demonstrate that DISCO correctly discriminates between algorithm quality: density-based methods outperform centroid-based ones on non-convex data, negative scores

meaningfully penalise harmful partitions, and near-equal scores on convex data confirm the metric is unbiased.

The package is available for installation from GitHub and is being prepared for CRAN submission.

References

[1] Beer, A., Krieger, L., Weber, P., Ritzert, M., Assent, I., Plant, C. (2025). *DISCO: Internal Evaluation of Density-Based Clustering.* arXiv:2503.00127 [cs.LG]. https://arxiv.org/abs/2503.00127

[2] Rousseeuw, P.J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.

[3] Ester, M., Kriegel, H.P., Sander, J., Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of KDD-96*, 226–231.

[4] Davies, D.L., Bouldin, D.W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2), 224–227.

[5] R Core Team (2023). *R: A Language and Environment for Statistical Computing.* R Foundation for Statistical Computing, Vienna, Austria. https://www.R-project.org

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.